

VISIÓN ARTIFICIAL · SISTEMAS EMBEBIDOS · INDUSTRIA 4.0

Implementación de OpenCV

Curso Taller de sistemas Embebidos

Elaborado por Álvaro Chacon Rosales 14/09/2026



¿Qué es OpenCV?



Código abierto

Licencia BSD. Reúne cientos de algoritmos de visión artificial, procesamiento de imágenes y aprendizaje automático dentro de un único framework.



Multilenguaje

Escrita originalmente en C++. Hoy expone interfaces de alto nivel para Python, C y Java sobre ese mismo núcleo nativo compilado.



Nota histórica

Citar la biblioteca correctamente en la literatura científica sostiene la credibilidad y la reproducibilidad del método. BibTeX de Gary Bradski, año 2000.

El núcleo del ecosistema de visión artificial abierta desde el año 2000.

El reto del mundo embebido e industrial

x86

La arquitectura para la que OpenCV fue diseñado originalmente

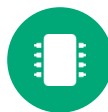
Computadoras de escritorio: memoria abundante, punto flotante nativo y consumo energético irrelevante. Nada de eso se cumple en un dispositivo embebido.

Cuellos de botella al migrar



Restricciones de memoria RAM

El framework completo no cabe: hay que compilar solo los módulos usados.



Compatibilidad de compiladores

Cadenas de compilación cruzada y bibliotecas del sistema que no coinciden.



Falta de punto flotante en hardware

Los procesadores de bajo costo carecen de soporte nativo; la emulación cuesta caro.



Ciclos de desarrollo acelerados

Menos margen para optimizar a mano cada algoritmo antes de salir a producción.

Cómo ven las imágenes las computadoras

01 Matrices numéricas

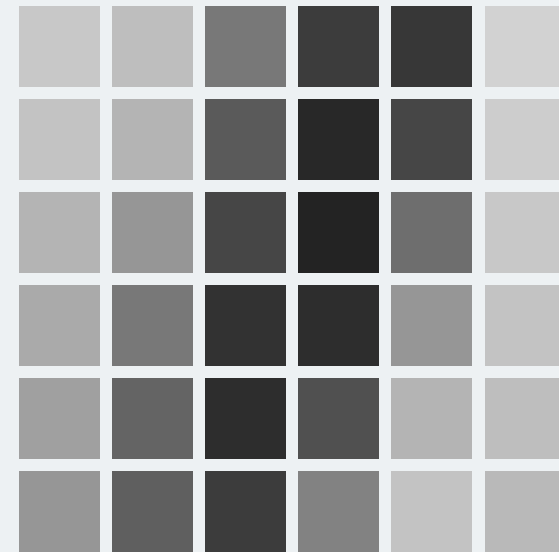
Toda imagen es un arreglo de números. En Python, literalmente un NumPy array que se puede indexar y operar.

02 Dos y tres dimensiones

Escala de grises = matriz bidimensional (2D). Color = matriz tridimensional (3D), un plano por canal.

03 El estándar BGR

Por defecto OpenCV interpreta el orden de color de los píxeles como BGR (Azul, Verde, Rojo), no como el clásico RGB.



Cada celda es un valor de intensidad de 0 a 255

ORDEN DE CANALES POR DEFECTO



Pipeline básico de visión artificial



Cada etapa reduce la cantidad de datos y aumenta el contenido semántico: de millones de píxeles a una decisión.

Detección de contornos y formas físicas



Transformada de Hough

Herramienta clave para aislar y detectar líneas rectas o círculos perfectos dentro de una imagen binaria, incluso con bordes incompletos.



Momentos e invariantes de Hu

Método matemático de orden central (M_{pq}) para extraer el área, el centro de gravedad y la orientación de un objeto.



Por qué importan en robótica y manufactura

Los descriptores de Hu son completamente invariantes ante traslación, escala y rotación del objeto: la pieza se reconoce sin importar cómo caiga sobre la banda transportadora.

Transparent API (T-API) y el uso de OpenCL

A partir de OpenCV 3.x se introdujo una API transparente que delega los cálculos a aceleradores (DSP o GPU) sin reescribir el algoritmo.

`cv::Mat`

- Memoria del host. La ejecución ocurre en la CPU.
- Control explícito de cada buffer.
- Sin delegación automática a aceleradores.

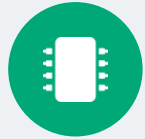


`cv::UMat`

- Universal Mat: sustituye a Mat en el código.
- Usa asignación de memoria compatible con OpenCL en tiempo de ejecución.
- El runtime decide si el cálculo va a CPU, GPU o DSP.

Misma función, distinto contenedor: el código de alto nivel no cambia y la portabilidad queda protegida.

Optimización en SoCs híbridos (ARM + DSP)



Procesador ARM

Ejecuta el sistema operativo, las comunicaciones, el control general y la lógica de aplicación.



DSP (TI C66x)

Asume la carga matemática pesada: punto flotante y procesamiento intensivo de señal.

Ping-Pong EDMA Framework

En lugar de procesar la imagen directamente sobre la DDR externa compartida, el acceso directo a memoria (DMA) mueve fragmentos (slices) hacia la memoria interna L2 del DSP, mucho más rápida. Mientras un slice se procesa, el siguiente ya se está transfiriendo: el cuello de botella de memoria desaparece.

Aceleración en hardware programable (FPGA y SoC)

100+

kernels optimizados basados en la biblioteca OpenCV



Plataformas AMD



Zynq · Zynq UltraScale+ · Versal · Alveo

Flujo de desarrollo en niveles

L1

Síntesis HLS directa

Kernels individuales sintetizados de forma directa mediante High-Level Synthesis, sin capa de host. Es el nivel donde se afina el rendimiento del hardware.

L2 / L3

Integración con host code

El código de aplicación en el procesador anfitrión se comunica con los kernels mediante OpenCL y el runtime XRT de AMD.

Prácticas críticas de diseño de software embebido



Gestión de memoria

Evitar la fragmentación dinámica preasignando bloques de almacenamiento estáticos (memory storage), calculados sobre el peor escenario posible de la aplicación.



Fugas de memoria (memory leaks)

Modificar punteros dentro de estructuras enlazadas dinámicas provoca caídas del sistema en operaciones continuas de 24/7, donde no hay reinicio que las disimule.



Estructuras lineales

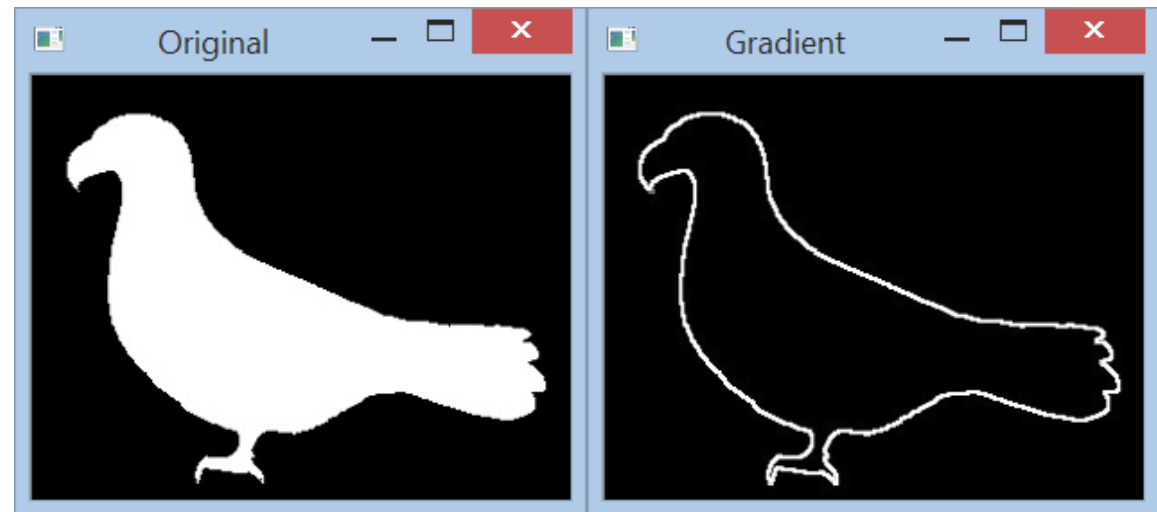
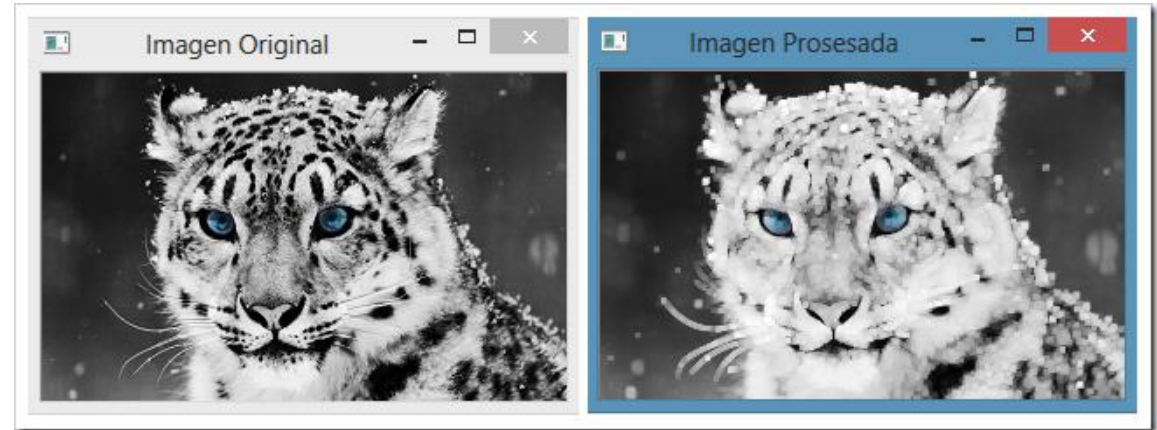
Simplificar el recorrido de contornos empleando listas lineales en lugar de jerarquías complejas: menos indirecciones, menos memoria y depuración más simple.

Operaciones con imágenes

```
img = cv.imread(filename)
```

```
_intensity = img[y,x]
```

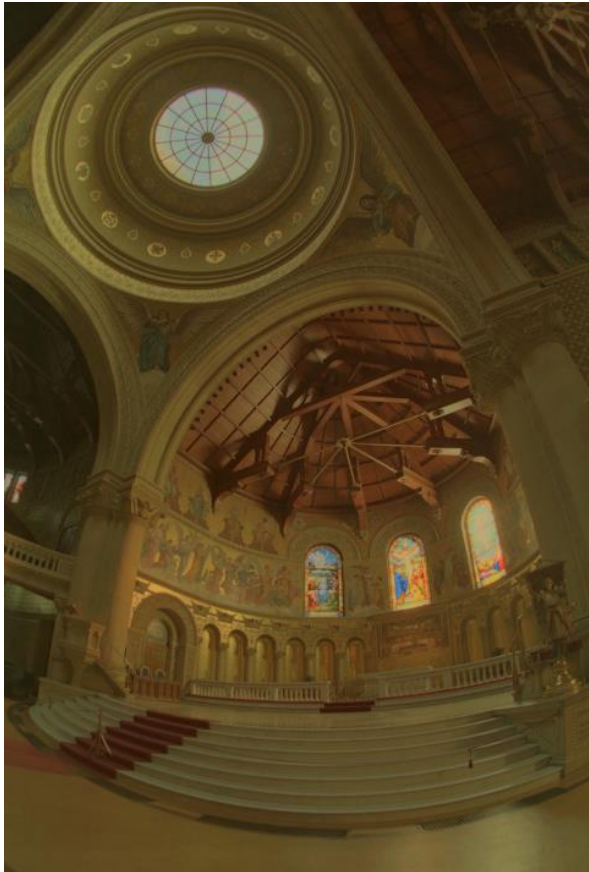
```
img = cv.imread('image.jpg')  
cv.namedWindow('image', cv.WINDOW_AUTOSIZE)  
cv.imshow('image', img)  
cv.waitKey()
```



Imágenes de alto rango dinámico

El proceso para combinar varias fotografías con diferentes exposiciones

Tonemapped image



Idea clave

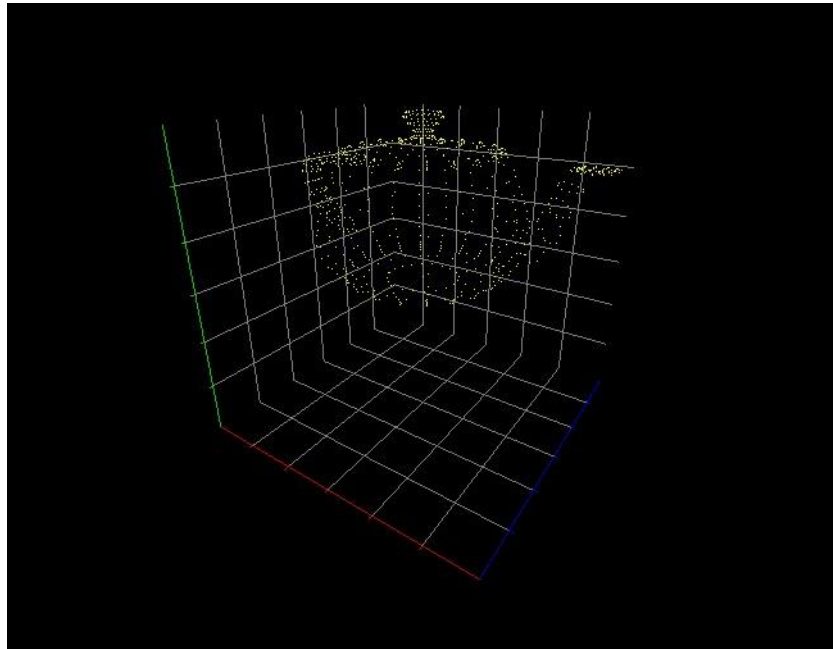
Una sola fotografía de 8 bits no siempre puede representar correctamente las zonas muy oscuras y muy brillantes de una escena

Exposure fusion

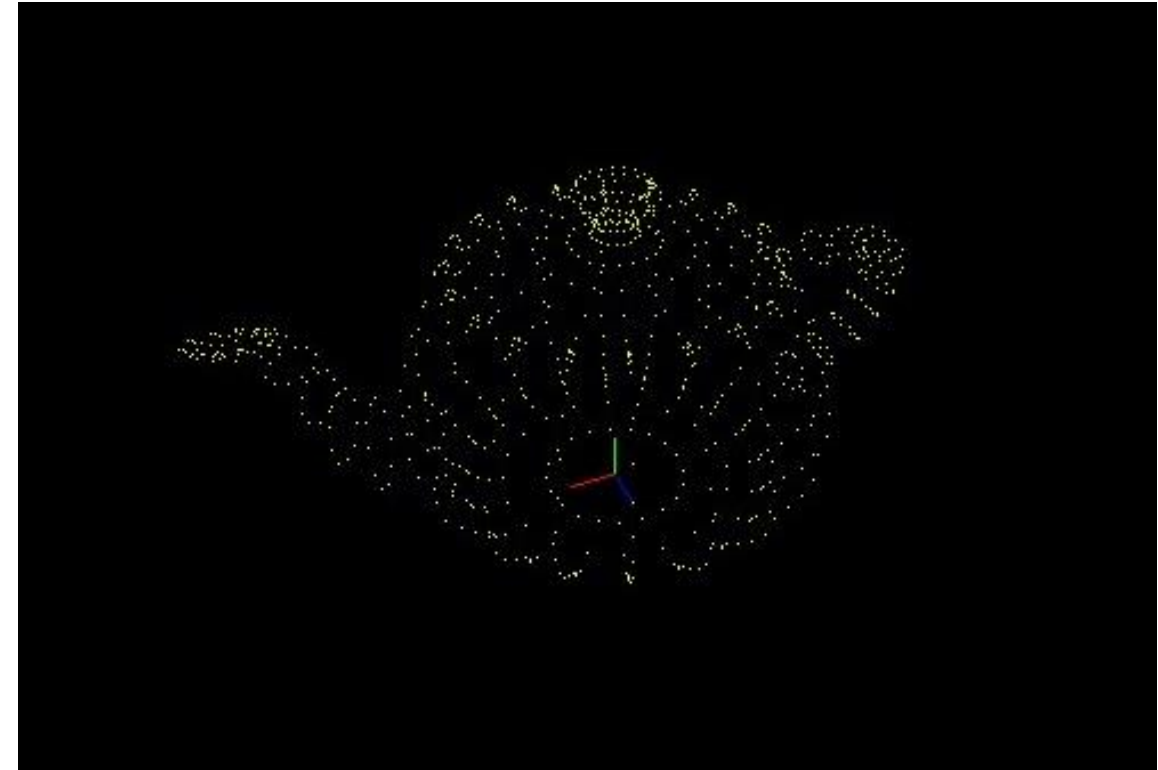


Visualización de nubes de puntos

consiste en representar un objeto o una escena mediante **muchísimos puntos en el espacio 3D**.

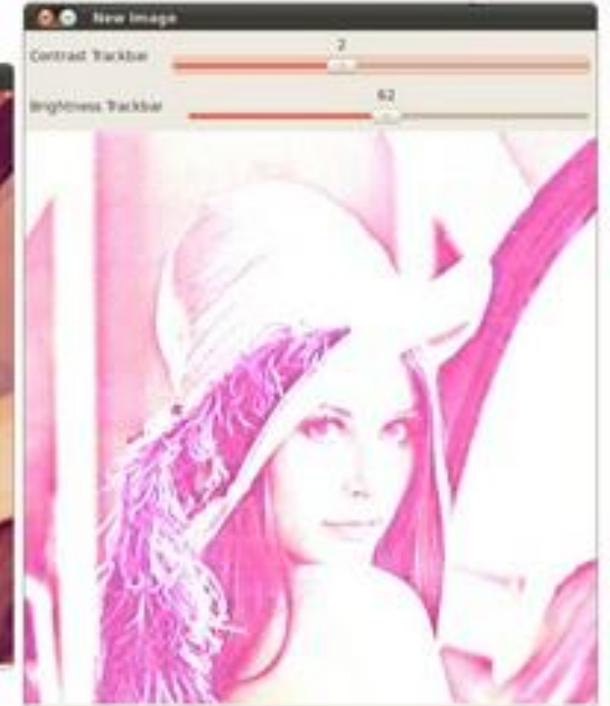
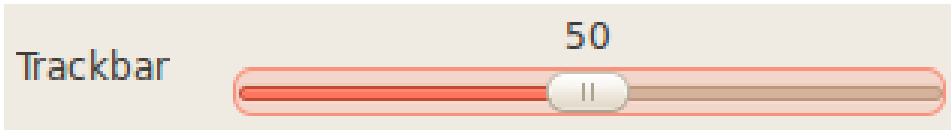


```
vertices, normals = cv2.loadPointCloud("teapot.obj")
```



Añadiendo una barra deslizante a nuestras aplicaciones

permitir que el usuario **cambie un valor en tiempo real** mientras el programa está funcionando.

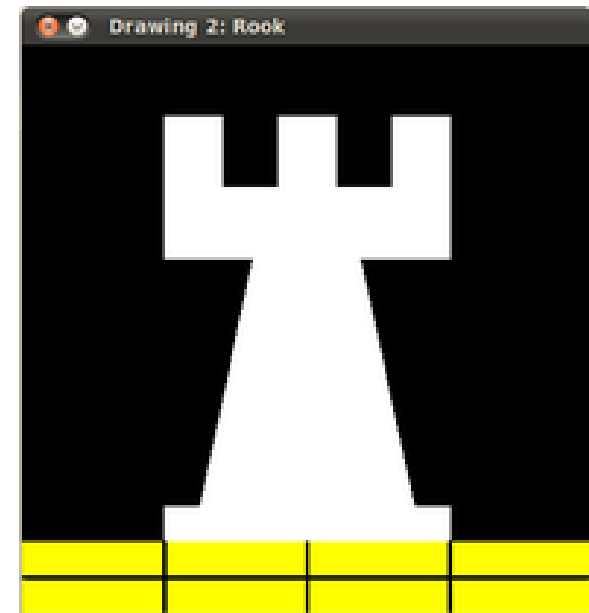
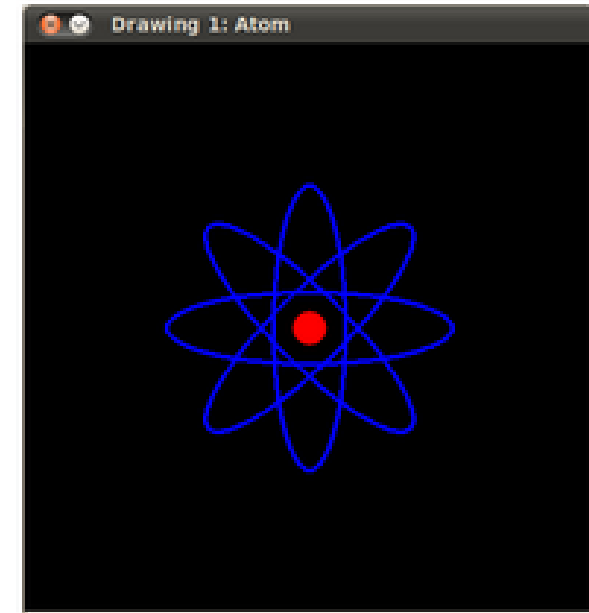


```
trackbar_name = 'Alpha x %d' % alpha_slider_max  
cv.createTrackbar(trackbar_name, title_window , 0, alpha_slider_max, on_trackbar)
```

Dibujo básico

Basic Drawing significa usar OpenCV para dibujar formas básicas directamente sobre una imagen.

Función	¿Qué dibuja?
<code>cv.line()</code>	Línea
<code>cv.rectangle()</code>	Rectángulo
<code>cv.circle()</code>	Círculo
<code>cv.ellipse()</code>	Elipse
<code>cv.polylines()</code>	Polígonos/líneas conectadas
<code>cv.putText()</code>	Texto



Metrología sin contacto en la Industria 4.0

EL PROBLEMA

Redondez y circularidad

Medir con precisión los errores de redondez en orificios de piezas maquinadas de DELRIN, sin desgastar el material.

LA SOLUCIÓN

Algoritmo MZC + Hough

Zona de Círculo Mínimo (Minimum Zone Circle) implementado en OpenCV junto con la Transformada de Hough.

HARDWARE

Cámara CMOS 5 MP · anillo LED · Raspberry Pi 4 · Arduino y motor a pasos

5 – 10 μm

de desviación frente a las costosas Máquinas de Medición por Coordenadas (CMM) de contacto

- Elimina el desgaste físico de la pieza
- Reduce drásticamente el tiempo de captura
- Sensores de adquisición mucho más baratos

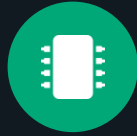


Comunicaciones IoT con el protocolo MQTT



OpenCV en Python

Ejecuta la medición y genera los datos dimensionales de la pieza.



Raspberry Pi 4

Actúa como broker local inalámbrico usando la librería Paho-MQTT.



Nube y clientes

Servidores, laptops y smartphones consumen los datos bajo una misma IP.



Ligero y desacoplado: el publicador no necesita saber quién consume el dato. Es lo que permite transmitir mediciones hacia la nube sobre enlaces de planta con ancho de banda limitado.

Guiado visual de robots e inspección automatizada



Arquitectura del sistema

- Plataforma de visión construida sobre OpenCV.
- Conexión por sockets TCP/IP síncronos hacia el controlador del robot.
- Robot industrial Adept Cobra s850 como actuador.
- Adquisición desde múltiples fuentes de vídeo simultáneas.



Evitación de colisiones en tiempo de ejecución

El algoritmo define Regiones de Interés (ROI) que corresponden exactamente a las huellas de los dedos de la pinza (impacket gripper).

El agarre (grasping) se autoriza únicamente si esas zonas están libres de obstáculos.

Conclusiones del análisis de las fuentes



01

Reducción drástica de costos

Las plataformas de visión artificial abiertas basadas en OpenCV llevan a cero el costo de licencia y permiten usar sensores de adquisición más económicos con un rendimiento comparable al del software comercial.



02

Interoperabilidad

El uso de estándares abiertos de comunicación (TCP, MQTT) y de formatos interpretados como STEP-NC permite que los recursos de la planta de producción sean conscientes de su entorno en tiempo real.



03

Portabilidad garantizada

El desacoplamiento entre el software de alto nivel y el hardware, mediante APIs estándar como OpenVX o la T-API de OpenCV, protege la inversión frente a la obsolescencia de los procesadores.

El futuro de la visión embebida



Aceleradores dedicados para CNN

Integración directa de modelos de aprendizaje profundo utilizando hardware especializado, como Hailo AI o el sensor Sony IMX500 sobre Raspberry Pi.



Arquitecturas híbridas

Migración de los servicios de procesamiento hacia esquemas híbridos que reparten la carga entre el borde y la computación en la nube.

Gracias

¿Preguntas?

Referencias:

- Bradski, G.** (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools. (Referencia fundamental de la biblioteca citada en la Slide 2).
- Răileanu, S., Borangiu, T., Anton, F., & Anton, S.** (2021). *Open source machine vision platform for manufacturing and robotics*. IFAC PapersOnLine, 54(1), 522–527. (Estudio de caso del robot Adept Cobra s850 citado en la Slide 13).
- Saif, Y., Yusof, Y., Rus, A. Z. M., Ghaleb, A. M., Mejjaouli, S., Al-Alimi, S., et al.** (2023). *Implementing circularity measurements in industry 4.0-based manufacturing metrology using MQTT protocol and Open CV: A case study*. PLoS ONE, 18(10), e0292814. (Caso de estudio del sistema de metrología 3SMVI y protocolo MQTT citado en la Slide 11).
- Raspberry Pi Ltd.** (2026). *The Picamera2 Library: A libcamera-based Python library for Raspberry Pi cameras*. Release 3. (Documentación y buenas prácticas de adquisición de video citadas en la Slide 14).
- Giduthuri, R. (Ed.).** (2020). *The OpenVX™ Specification*. Version 1.3. The Khronos OpenVX Working Group. (Definición del pipeline y el acumulador de la Transformada de Hough probabilística citada en la Slide 6).
- Prabhu, R.** (2011). *OpenCV on TI's C6000 DSP Architecture*. Texas Instruments Whitepaper. (Retos de compilación C++, gestión de memoria y migración embebida citados en las Slides 3, 8 y 10).
- Texas Instruments Incorporated.** (2021). *Processor SDK Linux for AM335X Documentation: 3.7. OpenCV*. (Aceleración T-API y estructura UMat con OpenCL citadas en la Slide 7).
- Advanced Micro Devices, Inc. (AMD).** (2026). *Vitis Vision Libraries: Application Development L1/L2/L3 workflows*. (Aceleración en hardware programable y FPGAs citada en la Slide 9).